

Подготовка к выполнению работы

Для выполнения практической работы использовалась таблица с котировками с индексами для быстрого получения информации по бумаге, дате котировке:

1) Составной индекс для быстрого получения данных по бумаге и дате - `CREATE INDEX idx_ticks_ticker_date ON ticks (ticker, date)`. Большинство запросов будут извлекать котировки конкретной бумаги за определенный период. Составной индекс позволит быстро найти все строки по тикеру и отсортировать их по дате.

2) Составной индекс для запросов по конкретной дате и времени. Если нужны запросы с точностью до времени, то этот индекс ускорит выборку.

```
CREATE TABLE ticks (  
    id SERIAL PRIMARY KEY,  
    ticker VARCHAR(10) NOT NULL,  
    per INTEGER NOT NULL CHECK (per >= 0),  
    date DATE NOT NULL,  
    time TIME NOT NULL,  
    open NUMERIC(10,2) NOT NULL CHECK (open > 0),  
    high NUMERIC(10,2) NOT NULL CHECK (high > 0),  
    low NUMERIC(10,2) NOT NULL CHECK (low > 0),  
    close NUMERIC(10,2) NOT NULL CHECK (close > 0),  
    vol INTEGER NOT NULL CHECK (vol >= 0),  
    CONSTRAINT high_ge_low CHECK (high >= low)  
)  
  
COPY ticks (ticker, per, date, time, open, high, low, close, vol)  
FROM '/home/staspy/Downloads/SBER_260101_260421.csv'  
DELIMITER ';' ;  
CSV HEADER;  
  
COPY ticks (ticker, per, date, time, open, high, low, close, vol)  
FROM '/home/staspy/Downloads/LKOH_260101_260421.csv'  
DELIMITER ';' ;  
CSV HEADER;  
  
CREATE INDEX idx_ticks_ticker_date ON ticks (ticker, date);  
CREATE INDEX idx_ticks_ticker_date_time ON ticks (ticker, date, time);
```

Ход выполнения работы

1) Составить структуру таблицы для представления рекурсивных данных, и запрос с использованием рекурсивного CTE (Таблица сделок, состоящая из полей: id операции, дата операции, бумага, кол-во (положительное или отрицательное в зависимости от покупки или продажи), ссылка на id операцию по покупке/продаже при предшествующей продаже/покупке). Рекурсивным запросом определить какая стратегия из цепочек сделок была более прибыльной

Была составлена структура таблицы для представления рекурсивных данных (Таблица сделок, состоящая из полей: id операции, дата операции, бумага, кол-во (положительное или отрицательное в зависимости от покупки или продажи), ссылка на id операцию по покупке/продаже при предшествующей продаже/покупке). В исходном условии price не указан, но для вычисления прибыльности цепочек сделок он необходим, поэтому столбец price и был добавлен. И столбец prev_trade_id необходим для ссылка на id операцию по покупке/продаже при предшествующей продаже/покупке, то есть для ссылки на операцию, которая была до текущей операции, чтобы мы могли удобно рассчитывать прибыль цепочки с помощью рекурсивного CTE в дальнейшем. Итоговый скрипт создания этой таблицы выглядит следующим образом:

```
CREATE TABLE trades (  
    id SERIAL PRIMARY KEY,  
    trade_date TIMESTAMP NOT NULL,  
    ticker VARCHAR(10) NOT NULL,  
    quantity INTEGER NOT NULL CHECK (quantity != 0),  
    price NUMERIC(10,2) NOT NULL CHECK (price > 0),  
    prev_trade_id INTEGER REFERENCES trades(id)  
);
```

Далее необходимо было подготовить данные для заполнения таблицы. Данными для этой таблицы будут являться различные цепочки операций, чтобы среди них мы могли выбрать самую прибыльную.

Первая цепочка:

Сначала покупаем 10 акций Сбербанка по 270, а потом продаем их по 295. Прибыль= $-10 \cdot 270 + 10 \cdot 295 = 250$ рублей. Продажа ссылается на покупку

```
-- 1 цепочка
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-01', 'SBER', 10, 270.00, NULL);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-10', 'SBER', -10, 295.00, 1);
```

Вторая цепочка:

Сначала покупаем 100 акций Лукойла по 150, потом продаем 40 штук по 160, затем продаем оставшиеся 60 по 170. Прибыль= $-100 \cdot 150 + 160 \cdot 40 + 170 \cdot 60 = 1600$. Продажи ссылаются на покупку.

```
-- 2 цепочка
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-03', 'LKOH', 100, 150.00, NULL);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-08', 'LKOH', -40, 160.00, 3);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-15', 'LKOH', -60, 170.00, 3);
```

Третья цепочка:

Сначала покупаем 15 акций Теслы по 285, затем покупаем еще 10 по 275, затем продаем все 25 по 292. Цепочка последовательная: покупка + покупка + продажа. Каждая сделка ссылается на предыдущую, образуя линейную структуру. Прибыль= $-15 \cdot 285 - 10 \cdot 275 + 25 \cdot 292 = 275$

```
-- 3 цепочка
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-06', 'TSLA', 15, 285.00, NULL);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-07', 'TSLA', 10, 275.00, 6);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-16', 'TSLA', -25, 292.00, 7);
```

Четвертая цепочка:

Эта цепочка очевидно будет с 0 прибылью, так как сначала купили 30 акций Роснефти по 400 рублей, а затем продали эти же 30 акций по 400 рублей. Продажа ссылается на покупку.

```
-- 4 цепочка
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-09', 'ROSN', 30, 400.00, NULL);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
```

```
VALUES ('2026-04-17', 'ROSN', -30, 400.00, 9);
```

Пятая цепочка:

Эту цепочку сделаем с убытком: покупка 20 акций ABCD (случайное название) по 5000 рублей, потом продажа по 4900 рублей. Прибыли нет, мы в минусе на -2000, так как $-20 \cdot 5000 + 4900 \cdot 20 = -2000$. Продажа ссылается на покупку.

```
-- 5 цепочка:
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-04', 'ABCD', 20, 5000.00, NULL);
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES ('2026-04-14', 'ABCD', -20, 4900.00, 11);
```

Шестая цепочка:

Сначала продажа 10 акций GOOD (случайное название) по 300 рублей, потом покупка 10 акций по 280 рублей. Прибыль=200, так как $10 \cdot 300 - 10 \cdot 280 = 200$. Покупка ссылается на продажу.

```
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)
VALUES
('2026-04-01', 'GOOD', -10, 300.00, NULL),
('2026-04-10', 'GOOD', 10, 280.00, 13);
```

Теперь когда таблица создана и тестовые данные вставлены, приступим к написанию рекурсивного CTE запроса, который найдет самую прибыльную цепочку:

```
WITH RECURSIVE trade_chain AS (
    SELECT
        id AS root_id,
        id AS current_id,
        ticker,
        quantity,
        price,
        prev_trade_id,
        0 AS level
    FROM trades
    WHERE prev_trade_id IS NULL
    UNION ALL
    SELECT
        tc.root_id,
        t.id,
        t.ticker,
        t.quantity,
        t.price,
        t.prev_trade_id,
        tc.level + 1
    FROM trades t
    JOIN trade_chain tc ON t.prev_trade_id = tc.current_id
```

```

)
SELECT
    root_id AS chain_id,
    ticker,
    SUM(
        CASE
            WHEN quantity < 0
            THEN -quantity * price
            ELSE -quantity * price
        END
    ) AS total_profit
FROM trade_chain
GROUP BY root_id, ticker
ORDER BY total_profit DESC
LIMIT 1;

```

Сначала в рекурсивном CTE отбираются все корни цепочек - записи, у которых prev_trade_id NULL. Для каждой такой записи сохраняется её id и как идентификатор всей цепочки (root_id), и как текущая сделка (current_id). Уровень у корневых записей получается нулевой. Затем к ним рекурсивно присоединяются все последующие сделки: каждая присоединяется, если её prev_trade_id совпадает с current_id предыдущей записи. При этом для каждой присоединённой сделки передаётся тот же самый root_id, а уровень увеличивается на один. Вот что хранится в CTE:

<pre> WITH RECURSIVE trade_chain AS (SELECT id AS root_id, id AS current_id, ticker, quantity, price, prev_trade_id, 0 AS level FROM trades WHERE prev_trade_id IS NULL UNION ALL SELECT tc.root_id, tc.current_id, tc.ticker, tc.quantity, tc.price, tc.prev_trade_id, tc.level + 1 FROM trades tc JOIN trade_chain ON tc.prev_trade_id = trade_chain.current_id) </pre>								
Results 1 x								
WITH RECURSIVE trade_chain AS (SELECT id AS Enter a SQL expression to filter results (use Ctrl+Space)								
Grid	☐	123 root_id	123 current_id	AZ ticker	123 quantity	123 price	123 prev_trade_id	123 level
Text	1	1	1	SBER	10	270	[NULL]	0
	2	3	3	LKOH	100	150	[NULL]	0
	3	6	6	TSLA	15	285	[NULL]	0
	4	9	9	ROSN	30	400	[NULL]	0
	5	11	11	ABCD	20	5,000	[NULL]	0
	6	13	13	GOOD	-10	300	[NULL]	0
	7	1	2	SBER	-10	295	1	1
	8	3	4	LKOH	-40	160	3	1
	9	3	5	LKOH	-60	170	3	1
	10	6	7	TSLA	10	275	6	1
	11	9	10	ROSN	-30	400	9	1
	12	11	12	ABCD	-20	4,900	11	1
	13	13	14	GOOD	10	280	13	1
	14	6	8	TSLA	-25	292	7	2

После того как все цепочки полностью собраны для каждой группы с одинаковым root_id и тикером суммируется результат. В расчёте участвуют все сделки цепочки, включая корневую. Для каждой сделки денежный поток вычисляется как $-\text{quantity} * \text{price}$: при отрицательном quantity (продажа) это даёт положительную сумму, при положительном quantity (покупка) - отрицательную. Итоговая прибыль или убыток это сумма таких потоков по всем сделкам цепочки. Остаётся отсортировать результат по убыванию прибыли и взять самую верхнюю строку - наиболее прибыльную цепочку.

The screenshot shows a SQL IDE interface. The top pane contains a SQL query using a recursive CTE. The query selects trade details and recursively builds a chain of trades starting from root trades (where prev_trade_id is NULL). The bottom pane shows the results of the query in a grid format. The first result row shows a chain_id of 1, a ticker of LKOH, and a total_profit of 1,600.

```
WITH RECURSIVE trade_chain AS (  
  SELECT  
    id AS root_id,  
    id AS current_id,  
    ticker,  
    quantity,  
    price,  
    prev_trade_id,  
    0 AS level  
  FROM trades  
  WHERE prev_trade_id IS NULL  
  UNION ALL  
  SELECT  
    tc.root_id,  
    t.id,  
    t.ticker,  
    t.quantity,  
    t.price,  
    tc.current_id AS prev_trade_id,  
    tc.level + 1 AS level  
  FROM trade_chain tc  
  JOIN trades t ON t.id = tc.current_id  
)
```

Results 1 x

WITH RECURSIVE trade_chain AS (SELECT id AS root_id, id AS current_id, ticker, quantity, price, prev_trade_id, 0 AS level FROM trades WHERE prev_trade_id IS NULL UNION ALL SELECT tc.root_id, t.id, t.ticker, t.quantity, t.price, tc.current_id AS prev_trade_id, tc.level + 1 AS level FROM trade_chain tc JOIN trades t ON t.id = tc.current_id)

	chain_id	ticker	total_profit
1	3	LKOH	1,600

Без LIMIT 1 результат каждой цепочки совпадает с тем какие значения прибыли мы получили при формировании тестовых данных:

The screenshot shows a SQL IDE interface. The top pane contains a SQL query:

```
SELECT
  root_id AS chain_id,
  ticker,
  SUM(
    CASE
      WHEN quantity < 0
      THEN -quantity * price
      ELSE -quantity * price
    END
  ) AS total_profit
FROM trade_chain
GROUP BY root_id, ticker
ORDER BY total_profit DESC
LIMIT 1;
```

The bottom pane shows the results of the query in a table format. The table has four columns: `chain_id`, `ticker`, and `total_profit`. The results are ordered by `total_profit` in descending order.

	chain_id	ticker	total_profit
1	3	LKOH	1,600
2	6	TSLA	275
3	1	SBER	250
4	13	GOOD	200
5	9	ROSN	0
6	11	ABCD	-2,000

2) Написать CTE запрос, который удалит записи в таблице котировок и вставит удаленные записи в аналогичную по столбцам таблицу логов.

Для выполнения этого запроса была создана таблица логов (`ticks_log`) для котировок (`ticks`) с аналогичной структурой, за исключением того, что были добавлены два новых столбца:

- `deleted_at` с типом данных `TIMESTAMP` и значение по умолчанию у которого равно `DEFAULT CURRENT_TIMESTAMP`. Поскольку `CURRENT_TIMESTAMP` вычисляется в момент вставки строки в таблицу логов, поле `deleted_at` фиксирует именно время попадания данных в лог, а не точное время выполнения операции `DELETE` в исходной таблице. Такое поведение обеспечивает автоматическую фиксацию

временной метки без необходимости явно указывать её в запросе. Стоит отметить, что есть возможность фиксировать время иным образом: например, прямо перед удалением занести его в переменную и вставлять ее в таблицу логов, или же написать триггер (AFTER DELETE, BEFORE INSERT), который будет заниматься этим. Так что всё зависит от требований: возможно нам необходимо чтобы фиксировалось именно время вставки, возможно нам необходимо чтобы фиксировалась именно время удаления. Оставим вариант с DEFAULT CURRENT_TIMESTAMP.

- deleted_by с типом данных VARCHAR(50) и значение по умолчанию у которого равно DEFAULT CURRENT_USER. Поскольку CURRENT_USER вычисляется в момент вставки строки в таблицу логов, поле deleted_by фиксирует именно пользователя, под которым был выполнен INSERT в таблицу логов, а не того, кто совершил DELETE в исходной таблице (хотя в рамках одного CTE-запроса это, как правило, один и тот же пользователь). Такой подход позволяет автоматически вести учет без передачи имени пользователя вручную.

Вот скрипт создания таблицы ticks_log:

```
CREATE TABLE ticks_log (  
  id INTEGER,  
  ticker VARCHAR(10) NOT NULL,  
  per INTEGER NOT NULL,  
  date DATE NOT NULL,  
  time TIME NOT NULL,  
  open NUMERIC(10,2) NOT NULL,  
  high NUMERIC(10,2) NOT NULL,  
  low NUMERIC(10,2) NOT NULL,  
  close NUMERIC(10,2) NOT NULL,  
  vol INTEGER NOT NULL,  
  deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  deleted_by VARCHAR(50) DEFAULT CURRENT_USER  
);
```

Далее был написан CTE-запрос, который удалил записи в таблице котировок и вставил удаленные записи в таблицу логов.

```
WITH deleted_ticks AS (  
  DELETE FROM ticks  
  WHERE  
    ticker = 'SBER' AND date BETWEEN '2026-01-01' AND '2026-04-01'  
  RETURNING *  
)
```

```
)  
INSERT INTO ticks_log (id, ticker, per, date, time, open, high, low, close,  
vol)  
SELECT id, ticker, per, date, time, open, high, low, close, vol  
FROM deleted_ticks  
RETURNING id, ticker, date, deleted_at;
```

В блоке WITH создаётся CTE с названием `deleted_ticks`, в котором выполняется удаление записей из таблицы `ticks`. В условии WHERE отбираются строки для удаления по тикеру `SBER` и дате в промежутке с 1 января по 1 апреля 2026 года. Через `RETURNING *` мы забираем все удалённые строки, и они сразу попадают в `deleted_ticks`, хотя из самой таблицы `ticks` эти данные уже удалены. Дальше выполняется `INSERT` в таблицу логов `ticks_log`, мы перечисляем все столбцы для вставки, кроме служебных полей `deleted_at` и `deleted_by`, потому что они проставляются сами через `DEFAULT`. В конце `RETURNING` показывает `id`, тикер, дату и `deleted_at` для каждой перенесённой записи, чтобы сразу видеть, что и когда было перемещено.

Стоит отметить, что CTE-подход гарантирует, что обе операции либо выполнятся полностью, либо не выполнятся вовсе. Если на этапе вставки в логи произойдёт сбой, то и удаление откатится.

Результаты:

Выполнение запроса:

●WITH deleted_ticks AS (
DELETE FROM ticks
WHERE
 ticker = 'SBER' AND date BETWEEN '2026-01-01' AND '2026-04-01'
RETURNING *
)
INSERT INTO ticks_log (id, ticker, per, date, time, open, high, low, close, vol)
SELECT id, ticker, per, date, time, open, high, low, close, vol
FROM deleted_ticks
RETURNING id, ticker, date, deleted_at;

ticks_log 1 ×

WITH deleted_ticks AS (DELETE FROM ticks WHERE Enter a SQL expression to filter results (use Ctrl+Space)

		123 id	AZ ticker	🕒 date	🕒 deleted_at	
Record	1	1	SBER	2026-01-05	2026-04-27 10:31:30.189	
	2	2	SBER	2026-01-05	2026-04-27 10:31:30.189	
	3	3	SBER	2026-01-05	2026-04-27 10:31:30.189	
	4	4	SBER	2026-01-05	2026-04-27 10:31:30.189	
	5	5	SBER	2026-01-05	2026-04-27 10:31:30.189	
	6	6	SBER	2026-01-05	2026-04-27 10:31:30.189	
	7	7	SBER	2026-01-05	2026-04-27 10:31:30.189	
	8	8	SBER	2026-01-05	2026-04-27 10:31:30.189	
	9	9	SBER	2026-01-05	2026-04-27 10:31:30.189	
	10	10	SBER	2026-01-05	2026-04-27 10:31:30.189	
	11	11	SBER	2026-01-05	2026-04-27 10:31:30.189	
	12	12	SBER	2026-01-05	2026-04-27 10:31:30.189	
	13	13	SBER	2026-01-05	2026-04-27 10:31:30.189	
	14	14	SBER	2026-01-05	2026-04-27 10:31:30.189	

Запрос к ticks_log для проверки, что вставка была выполнена:

SELECT * FROM ticks_log;

ticks_log 1 ×

SELECT * FROM ticks_log; Enter a SQL expression to filter results (use Ctrl+Space)

		123 id	AZ ticker	123 per	🕒 date	🕒 time	123 open	123 high	123 low	123 close	123 vol	🕒 deleted_at	AZ deleted_by
Record	1	1	SBER	1	2026-01-05	07:00:00	299.99	300.12	299.3	299.77	91,164	2026-04-27 10:31:30.189	postgres
	2	2	SBER	1	2026-01-05	07:01:00	299.61	299.77	299.22	299.47	71,609	2026-04-27 10:31:30.189	postgres
	3	3	SBER	1	2026-01-05	07:02:00	299.47	299.47	299.1	299.16	43,542	2026-04-27 10:31:30.189	postgres
	4	4	SBER	1	2026-01-05	07:03:00	299.25	299.26	298.96	299.2	122,998	2026-04-27 10:31:30.189	postgres
	5	5	SBER	1	2026-01-05	07:04:00	299.02	299.16	299.01	299.08	32,275	2026-04-27 10:31:30.189	postgres
	6	6	SBER	1	2026-01-05	07:05:00	299.06	299.25	299.06	299.15	15,729	2026-04-27 10:31:30.189	postgres
	7	7	SBER	1	2026-01-05	07:06:00	299.21	299.26	299.14	299.14	8,897	2026-04-27 10:31:30.189	postgres
	8	8	SBER	1	2026-01-05	07:07:00	299.14	299.22	299.12	299.13	6,872	2026-04-27 10:31:30.189	postgres
	9	9	SBER	1	2026-01-05	07:08:00	299.13	299.13	298.87	298.9	27,270	2026-04-27 10:31:30.189	postgres
	10	10	SBER	1	2026-01-05	07:09:00	298.9	298.99	298.88	298.98	7,164	2026-04-27 10:31:30.189	postgres
	11	11	SBER	1	2026-01-05	07:10:00	298.98	299.01	298.96	298.99	14,815	2026-04-27 10:31:30.189	postgres
	12	12	SBER	1	2026-01-05	07:11:00	299	299.2	298.99	299.14	75,633	2026-04-27 10:31:30.189	postgres
	13	13	SBER	1	2026-01-05	07:12:00	299.2	299.22	298.77	298.98	51,319	2026-04-27 10:31:30.189	postgres
	14	14	SBER	1	2026-01-05	07:13:00	299.05	299.17	299.05	299.17	384	2026-04-27 10:31:30.189	postgres
	15	15	SBER	1	2026-01-05	07:14:00	299.17	299.21	299.16	299.21	2,796	2026-04-27 10:31:30.189	postgres

Запрос к ticks для проверки, что удаление точно выполнено:

The screenshot shows a SQL IDE with a query editor at the top containing the following SQL statement:

```
SELECT * FROM ticks
WHERE ticker = 'SBER' AND date BETWEEN '2026-01-01' AND '2026-04-01';
```

Below the query editor, the results are displayed in a table with 11 columns. The first column is labeled 'id' and the others are 'ticker', 'per', 'date', 'time', 'open', 'high', 'low', 'close', and 'vol'. The table is currently empty.

id	ticker	per	date	time	open	high	low	close	vol

3) Задача на ранжирование. Найти недели в котировках, где были максимальные обороты в течении недели, выбрать 3 недели по убываю с максимальными оборотами, вывести для них котировки по дням.

Для начала напишем CTE-запрос, который вычисляет обороты по каждой неделе, то есть недельные обороты:

```
WITH weekly_volumes AS (
    SELECT
        ticker,
        DATE_TRUNC('week', date)::DATE AS week_start,
        (DATE_TRUNC('week', date) + INTERVAL '6 days')::DATE AS week_end,
        SUM(vol) AS week_total_vol
    FROM ticks
    GROUP BY ticker, DATE_TRUNC('week', date)
),
```

DATE_TRUNC('week', date) - это функция, которая округляет дату вниз до начала недели. Она возвращает TIMESTAMP, поэтому необходимо явно преобразовать к DATE.

(DATE_TRUNC('week', date) + INTERVAL '6 days')::DATE AS week_end - а так мы получаем конец недели - воскресенье: DATE_TRUNC('week', date) берёт дату и округляет её вниз до начала недели (понедельника), возвращая timestamp (с временем 00:00:00). Затем к этому timestamp прибавляется INTERVAL '6 days' - получаем воскресенье этой же недели, тоже timestamp (00:00:00). И ::DATE преобразует timestamp обратно в чистую дату, отбрасывая время.

Таким образом мы получаем начало и конец недели и теперь можем посчитать недельные обороты каждой недели используя GROUP BY с функцией SUM.

Далее напишем вторую часть CTE-запроса - ранжируем недели по убыванию оборотов. Для этого используем оконную функцию ранжирования RANK() создавая окно по столбцу с недельными оборотами по убыванию - то есть чтобы неделя с самым большим оборотом имела ранг 1. RANK() ранжирует с пропусками, поэтому выбираем DENSE_RANK() чтобы получить топ-3 значения оборота. При ничьих получаем больше недель, но гарантированно 3 разных уровня оборотов.

```
top_weeks AS (  
    SELECT  
        ticker,  
        week_start,  
        week_end,  
        week_total_vol,  
        DENSE_RANK() OVER (ORDER BY week_total_vol DESC) AS week_rank  
    FROM weekly_volumes  
)
```

Теперь предоставлю следующий скрипт полного запроса, где основной запрос извлекает котировки для трёх недель с максимальными торговыми оборотами. Для каждого дня внутри выбранных недель вычисляются агрегированные показатели: максимальная цена (max(high)), минимальная цена (min(low)), средняя цена закрытия (avg(close)) и суммарный объём торгов (sum(vol)). Группировка выполняется по тикеру, дате, началу и концу недели, общему недельному объёму и рангу недели - это необходимо, чтобы вывести все неагрегированные столбцы из CTE top_weeks в результирующем наборе, при этом сохранив уникальность строк по дням для каждого тикера. Соединение с CTE top_weeks происходит по тикеру и диапазону дат, ограничивая выборку неделями с рангом не выше трёх. Результат сортируется по рангу недели, тикеру и дате.

```
WITH weekly_volumes AS (  
    SELECT  
        ticker,  
        DATE_TRUNC('week', date)::DATE AS week_start,  
        (DATE_TRUNC('week', date) + INTERVAL '6 days')::DATE AS week_end,  
        SUM(vol) AS week_total_vol  
    FROM ticks  
    GROUP BY ticker, DATE_TRUNC('week', date)  
) ,  
  
top_weeks AS (  
    SELECT
```

```

        ticker,
        week_start,
        week_end,
        week_total_vol,
        DENSE_RANK() OVER (ORDER BY week_total_vol DESC) AS week_rank
    FROM weekly_volumes
)

SELECT
    t.ticker,
    t.date,
    max(t.high),
    min(t.low),
    avg(t.close),
    sum(t.vol),
    tw.week_start,
    tw.week_end,
    tw.week_total_vol,
    tw.week_rank
FROM ticks t
JOIN top_weeks tw
    ON t.ticker = tw.ticker
    AND t.date BETWEEN tw.week_start AND tw.week_end
WHERE tw.week_rank <= 3
GROUP BY t.ticker, t.date, tw.week_start,
    tw.week_end,
    tw.week_total_vol,
    tw.week_rank
ORDER BY tw.week_rank, t.ticker, t.date;

```

Примечание: технически группировка по `tw.week_start`, `tw.week_end`, `tw.week_total_vol` и `tw.week_rank` избыточна относительно группировки по `t.ticker` и `t.date` (так как они однозначно определяются неделями), но требуется синтаксически, потому что эти столбцы не агрегированы в `SELECT`.

Пример выполнения запроса:

The screenshot shows a SQL IDE with a query editor and a results grid. The query is as follows:

```
WITH weekly_volumes AS (  
    SELECT  
        ticker,  
        DATE_TRUNC('week', date)::DATE AS week_start,  
        (DATE_TRUNC('week', date) + INTERVAL '6 days')::DATE AS week_end,  
        SUM(vol) AS week_total_vol  
    FROM ticks  
    GROUP BY ticker, DATE_TRUNC('week', date)  
),  
top_weeks AS (  
    SELECT  
        ticker,  
        week_start,  
        week_end,  
        week_total_vol,  
        DENSE_RANK() OVER (ORDER BY week_total_vol DESC) AS week_rank  
    FROM weekly_volumes  
),  
SELECT  
    t.ticker,  
    t.date,  
    max(t.high),  
    min(t.low),  
    avg(t.close),  
    sum(t.vol),  
    tw.week_start,
```

The results grid shows data for the week of 2026-02-16 to 2026-02-22. The columns are: Az ticker, date, l23 max, l23 min, l23 avg, l23 sum, week_start, week_end, l23 week_total_vol, and l23 week_rank. The data is as follows:

	Az ticker	date	l23 max	l23 min	l23 avg	l23 sum	week_start	week_end	l23 week_total_vol	l23 week_rank
1	SBER	2026-02-16	312.4	308.84	310.825959799	38,774,229	2026-02-16	2026-02-22	132,263,775	1
2	SBER	2026-02-17	312	308.86	310.6060100503	21,194,399	2026-02-16	2026-02-22	132,263,775	1
3	SBER	2026-02-18	313.96	309.54	312.3301105528	34,607,362	2026-02-16	2026-02-22	132,263,775	1
4	SBER	2026-02-19	314.77	312.33	313.4043718593	18,992,778	2026-02-16	2026-02-22	132,263,775	1
5	SBER	2026-02-20	314.59	312.12	313.5903115578	15,826,360	2026-02-16	2026-02-22	132,263,775	1
6	SBER	2026-02-21	314.29	313.05	313.6588170055	1,315,016	2026-02-16	2026-02-22	132,263,775	1
7	SBER	2026-02-22	314.67	313.53	314.0008502773	1,553,631	2026-02-16	2026-02-22	132,263,775	1

4) Написать хранимую функцию, которая в цикле вычисляет сумму портфеля на конец месяца по таблице из пункта 1 и складывает результат в отдельную таблицу с обработкой исключений; в случае возникновения исключения нужно делать запись в журнал.

Так как в таблице из пункта 1 содержались только сделки апреля 2026, то добавим для наглядности еще сделки мая 2026 и июня 2026:

```
-- Май 2026  
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)  
VALUES  
( '2026-05-05', 'SBER', 20, 300.00, NULL),  
( '2026-05-10', 'SBER', -5, 310.00, 15),  
( '2026-05-12', 'LKOH', 50, 155.00, NULL),  
( '2026-05-15', 'ROSN', -10, 420.00, 9),  
( '2026-05-20', 'VTBR', 100, 95.00, NULL);  
  
-- Июнь 2026  
INSERT INTO trades (trade_date, ticker, quantity, price, prev_trade_id)  
VALUES  
( '2026-06-01', 'SBER', -15, 305.00, 15),  
( '2026-06-05', 'LKOH', -30, 165.00, 17),  
( '2026-06-10', 'ROSN', -20, 410.00, 9),  
( '2026-06-15', 'VTBR', -50, 100.00, 20),  
( '2026-06-20', 'SBER', 10, 290.00, NULL);
```

Скрипт для создания таблицы где хранится сумма портфеля (portfolio_monthly) на конец месяца и таблицы-журнала (error_log):

```
CREATE TABLE portfolio_monthly (  
    id SERIAL PRIMARY KEY,  
    ticker VARCHAR(10) NOT NULL,  
    month_end_date DATE NOT NULL,  
    total_quantity INTEGER NOT NULL,  
    portfolio_sum NUMERIC(10, 2) NOT NULL,  
    portfolio_profit NUMERIC(10,2) NOT NULL,  
    calculated_at TIMESTAMP DEFAULT NOW(),  
    UNIQUE(ticker, month_end_date)  
);  
  
CREATE TABLE error_log (  
    id SERIAL PRIMARY KEY,  
    error_date TIMESTAMP DEFAULT NOW(),  
    error_message TEXT,  
    error_detail TEXT,  
    ticker VARCHAR(10),  
    month_date DATE  
);
```

Для расчёта реальной суммы портфеля нужна текущая рыночная цена бумаги на конец месяца, поэтому пусть эта актуальная рыночная цена бумаги будет храниться в другой таблице (current_tickers_prices). Цены сформировал сам:

```
CREATE TABLE current_tickers_prices (  
    ticker VARCHAR(10) PRIMARY KEY,  
    price NUMERIC(10,2) NOT NULL  
)  
  
INSERT INTO current_tickers_prices (ticker, price) VALUES  
( 'ABCD', 4950.00),  
( 'GOOD', 290.00),  
( 'LKOH', 160.00),  
( 'ROSN', 415.00),  
( 'SBER', 295.00),  
( 'TSLA', 280.00),  
( 'VTBR', 98.00);
```

Функция, которая в цикле вычисляет сумму портфеля на конец месяца по таблице из пункта 1 и складывает результат в отдельную таблицу с обработкой исключений; в случае возникновения исключения нужно делать запись в журнал:

```
CREATE OR REPLACE FUNCTION calculate_portfolio_monthly(  
    p_month INTEGER,  
    p_year INTEGER  
)  
RETURNS VOID
```



```

LANGUAGE plpgsql
AS $$
DECLARE
    month_end DATE;
    ticker_record RECORD;
    total_qty INTEGER;
    total_cost NUMERIC(10,2);
    current_price NUMERIC(10,2);
    vportfolio_sum NUMERIC(10,2);
BEGIN
    IF p_month IS NULL OR p_year IS NULL THEN
        RAISE EXCEPTION 'Месяц и год не могут быть NULL';
    END IF;

    IF p_month < 1 OR p_month > 12 THEN
        RAISE EXCEPTION 'Месяц должен быть от 1 до 12, передан: %', p_month;
    END IF;

    month_end := (DATE_TRUNC('month', MAKE_DATE(p_year, p_month, 1)) +
INTERVAL '1 month' - INTERVAL '1 day')::DATE;

    RAISE NOTICE 'Расчёт портфеля на %', month_end;

    FOR ticker_record IN
        SELECT DISTINCT ticker
        FROM trades
        WHERE trade_date::DATE <= month_end
        ORDER BY ticker
    LOOP
        BEGIN
            SELECT COALESCE(SUM(quantity), 0)
            INTO total_qty
            FROM trades
            WHERE ticker = ticker_record.ticker
            AND trade_date::DATE <= month_end;

            SELECT -COALESCE(SUM(quantity * price), 0)
            INTO total_cost
            FROM trades
            WHERE ticker = ticker_record.ticker
            AND trade_date::DATE <= month_end;

            SELECT price
            INTO current_price
            FROM current_tickers_prices
            WHERE ticker = ticker_record.ticker;

            vportfolio_sum := total_qty * current_price;

            INSERT INTO portfolio_monthly (ticker, month_end_date,
total_quantity, portfolio_sum, portfolio_profit)
                VALUES (ticker_record.ticker, month_end, total_qty,
vportfolio_sum, total_cost);

```

```

        EXCEPTION
        WHEN OTHERS THEN
            INSERT INTO error_log (error_message, error_detail, ticker,
month_date)
                VALUES (SQLERRM, SQLSTATE, ticker_record.ticker, month_end);
            CONTINUE;

        END;
    END LOOP;

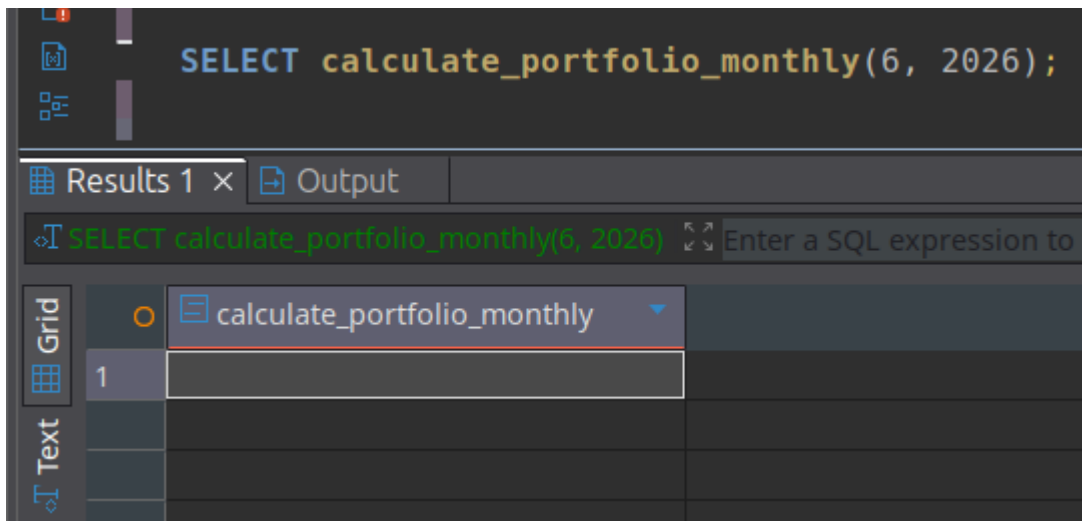
    RAISE NOTICE 'Расчёт завершён';

EXCEPTION
    WHEN OTHERS THEN
        INSERT INTO error_log (error_message, error_detail, ticker,
month_date)
            VALUES ('Критическая ошибка: ' || SQLERRM, SQLSTATE, NULL, NULL);
END;
$$;

```

Функция принимает на вход месяц и год, вычисляет состояние портфеля на последний день указанного месяца и сохраняет результат в таблицу portfolio_monthly. В цикле обходятся все уникальные тикеры, по которым были сделки до указанной даты. Для каждого тикера суммируется общее количество оставшихся бумаг и вычисляется финансовый результат по всем совершённым операциям - portfolio_profit, представляющий собой сумму произведений количества на цену сделки с учётом знака, что отражает накопленную прибыль или убыток по бумаге. Текущая рыночная цена берётся из таблицы current_tickers_prices, и на её основе рассчитывается сумма портфеля portfolio_sum как произведение остатка бумаг на эту цену. При возникновении ошибки по какому-либо тикеру выполняется запись в журнал error_log, после чего цикл продолжает обработку следующих тикеров. Предусмотрен также глобальный перехват исключений для критических ошибок всей функции.

Пример работы правильного использования функции:



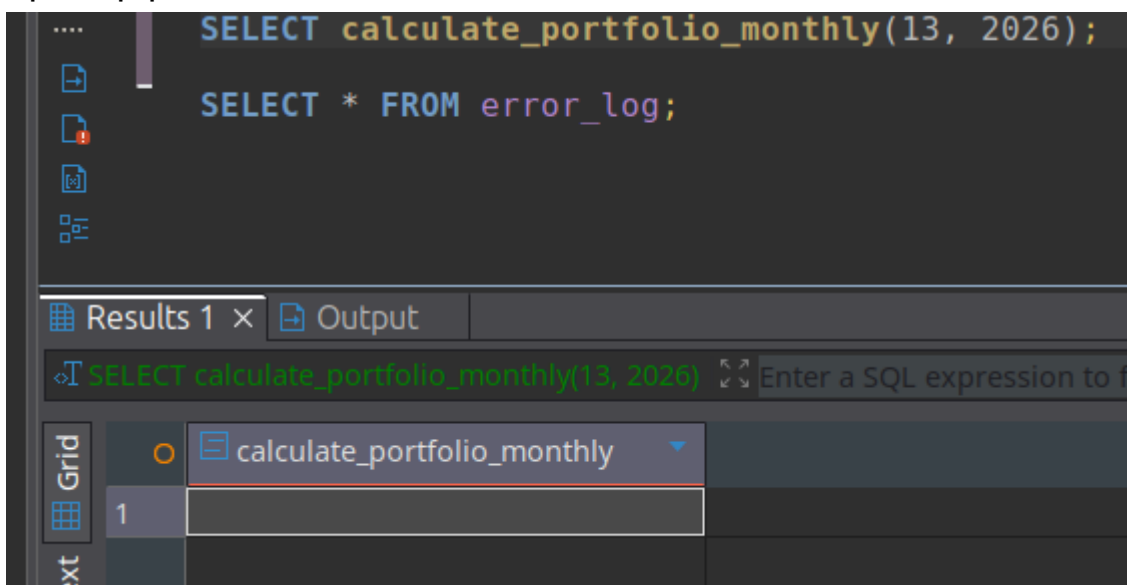
SELECT calculate_portfolio_monthly(4, 2026);
SELECT calculate_portfolio_monthly(6, 2026);
SELECT * FROM portfolio_monthly ORDER BY month_end_date;

portfolio_monthly 1 × Output

SELECT * FROM portfolio_monthly ORDER BY month_end_date

	id	AZ ticker	month_end_date	total_quantity	portfolio_sum	portfolio_profit	calculated_at
1	8	ABCD	2026-04-30	0	0	-2,000	2026-04-29 14:18:25.272
2	9	GOOD	2026-04-30	0	0	200	2026-04-29 14:18:25.272
3	10	LKOH	2026-04-30	0	0	1,600	2026-04-29 14:18:25.272
4	11	ROSN	2026-04-30	0	0	0	2026-04-29 14:18:25.272
5	12	SBER	2026-04-30	0	0	250	2026-04-29 14:18:25.272
6	13	TSLA	2026-04-30	0	0	275	2026-04-29 14:18:25.272
7	1	ABCD	2026-06-30	0	0	-2,000	2026-04-29 14:18:04.658
8	7	VTBR	2026-06-30	50	4,900	-4,500	2026-04-29 14:18:04.658
9	2	GOOD	2026-06-30	0	0	200	2026-04-29 14:18:04.658
10	3	LKOH	2026-06-30	20	3,200	-1,200	2026-04-29 14:18:04.658
11	4	ROSN	2026-06-30	15	6,225	-6,250	2026-04-29 14:18:04.658
12	5	SBER	2026-06-30	10	2,950	-2,525	2026-04-29 14:18:04.658
13	6	TSLA	2026-06-30	0	0	275	2026-04-29 14:18:04.658

Пример работы с ошибкой:



SELECT * FROM error_log;

error_log 1 x

Output

SELECT * FROM error_log

Enter a SQL expression to filter results (use Ctrl+Space)

	id	error_date	error_message	error_detail	ticker	month_date
1	1	2026-04-29 14:20:15.385	Критическая ошибка: Месяц должен быть от 1 до 12, передан: 13	P0001	[NULL]	[NULL]